

Climate Mesh

A decentralised, explainable climate early-warning mesh on a Raspberry Pi 5

Luis Yu and Leo Zhang · Years 12-13

PA Raspberry Pi Competition 2026/27 · Theme: Building a Positive Human Future (Safer Societies & Sustainable World)

1. The problem

Flood and heat warnings fail exactly where they matter. The official river gauge nearest to a school or an estate can be kilometres away from the stream, drain or underpass that actually floods, so by the time an official warning reaches a community the water is already rising. The World Meteorological Organization reports that countries with limited early-warning coverage suffer disaster mortality nearly eight times higher than those with substantial coverage (WMO, *Global Status of Multi-Hazard Early Warning Systems*, 2023). There is a gap between where climate events happen and where they are measured. We wanted a low-cost way for a school or a community to watch its own streets, and to be told in plain English what to do.

2. What we built

Climate Mesh is a network of 20 environmental “nodes” across Greater London that runs on a single Raspberry Pi 5, fully offline, with no cloud account and no API key. Every two seconds each node reports eight channels: temperature, humidity, air quality, water level, wind speed, wind chill, heat index and barometric pressure. A risk engine scores every location from 0 to 100, an explainable AI model flags unusual combinations of readings, *mesh correlation* checks whether neighbouring nodes agree before escalating, and plain-English alerts with community action playbooks appear on a seven-tab dashboard. Judges can prove the whole system works from a clean copy with one command.



Figure 1 — The Raspberry Pi 5 running Climate Mesh. Photo: the team.

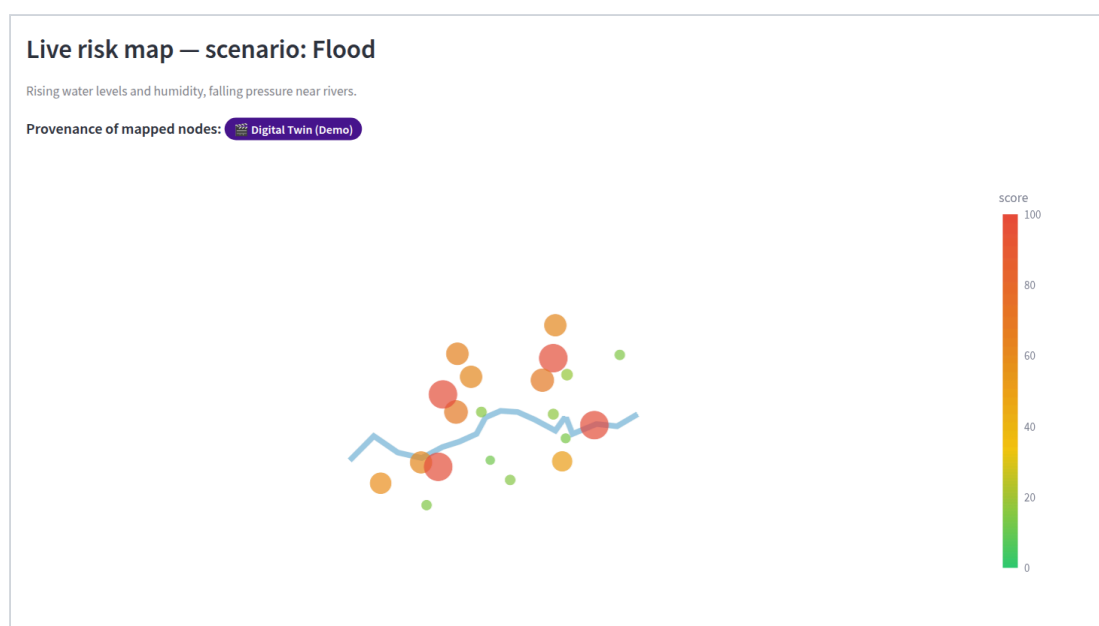


Figure 2 — Live Map, flood scenario, demo (simulated) data. Marker colour and size show risk 0–100; the river nodes escalate first.

3. How it works

One reading shape for every source. Whether a value comes from a physical sensor, the live Open-Meteo weather service or our simulator, it is turned into the same canonical reading: the eight channels plus a source label (*hardware / api / demo / simulation*) and a *quality_flag* (*ok / estimated / stale / missing*). Everything downstream, the risk engine, the AI, the database, the dashboard and the evidence export, only ever sees that shape. A real sensor joins the mesh by adding one adapter; nothing else changes.

An explainable risk score. For each node the engine computes six hazard sub-scores on a 0–100 scale (temperature, humidity, air quality, water level, wind, pressure) and combines them as *worst hazard plus 20 % of the rest*, so one severe hazard alone can reach CRITICAL while several moderate ones also escalate. The bands are SAFE (under 30),

MODERATE (30–60), WARNING (60–80) and CRITICAL (80–100). Temperature and humidity are two-sided: a frosty morning is filed as a *cold* hazard with its own advice, never as a “heatwave”.

AI that explains itself. An Isolation Forest (120 trees) learns the normal multivariate shape of the data and flags readings that are easy to isolate, catching an unusual *combination* of values before any single channel crosses a hard limit. Offline it trains on 2,000 deterministic synthetic samples; when online it can instead train on about 30 days of real hourly ERA5 weather for London from the Open-Meteo archive, and it records which it used. A confirmed anomaly multiplies the node’s risk by up to 1.5×, and the model reports which channels deviate most from the learned baseline, so every alert says *why*.

Mesh correlation as trust. Two nodes within 6 km are neighbours. The 1.2× mesh multiplier fires only when a node *and* at least two of its neighbours are elevated for the *same* hazard. A single glitching sensor cannot cry wolf; a correlated regional event is escalated.

Alerts people can act on. WARNING and CRITICAL results raise an alert containing the plain-English explanation and a playbook of practical, low-risk actions for that hazard (flood, heatwave, smog, storm, cold). Alerts are rate-limited so the log never fills with duplicates.



Figure 3 — Node Detail, Regent’s Canal, flood scenario, demo data: the “Why:” sentence and the six sub-scores behind a 100/100 score.

Architecture. The engine process reads the nodes and scores them; a SQLite database in write-ahead-log mode is the message bus; the Streamlit dashboard is a pure reader of that database. A one-command export writes every reading, score and alert to CSV and JSON, each row still carrying its source and quality flag, plus a table of how many readings came from each source.

4. What is genuinely new

1. **Sensor-ready without being sensor-dependent.** The canonical reading contract means the system is complete today on simulated and live-API data, and a physical node is a drop-in, not a rewrite.
2. **Mesh correlation as trust.** Escalation needs agreement between adjacent nodes for the same hazard, which is how a network of cheap nodes can be more

trustworthy than one expensive one.

3. **Provenance-first honesty.** Every reading, dashboard panel, alert and exported row carries its data source and a quality flag. The dashboard can only show a “Physical Sensor” badge when a reading really came from a device. Honesty is a designed-in feature, not a disclaimer.
4. **One-command reproducibility.** `python scripts/judge_validate.py` runs the smoke test, all 166 automated tests, a normal and a flood demo cycle and the evidence export, and prints a PASS/FAIL table.

5. What is real and what is simulated

We label exactly what this is and is not.

Mode	Data	Internet	Sensors
simulation	Realistic generated data (default)	No	No
demo	Deterministic, screenshot-stable data	No	No
api	Live Open-Meteo weather and air quality for all 20 locations	Yes, falls back to simulation	No
hardware	One physical Vernier weather sensor, simulated mesh for the rest	No	Yes, falls back if absent
auto	Hardware, else API, else simulation	Optional	Optional

- The 20 nodes sit at well-known London landmarks chosen to exercise the environment types (river, residential, urban, park). They are illustrative, not deployment sites.
- The hardware path (a Vernier Go Direct Weather sensor over USB) is implemented and unit-tested for its fallback and labelling behaviour, but it has **not yet been validated against a physical device**. A node is labelled hardware only after a device actually opens and returns a reading.
- The flood, heatwave, smog and storm scenarios are simulated so that judging never depends on a real emergency. In api and hardware modes live values are shown unmodified.
- The AI is decision support for a community, not an accredited warning system. Alerts always defer to official guidance.

6. Evidence that it works

- **166 automated tests pass** on Python 3.11, 3.12 and 3.13, including a test that renders all seven dashboard tabs in a browser harness and one that pins the exact demo numbers quoted below.

- `python scripts/judge_validate.py` → **PASS (5 of 5 steps)**.
- `python scripts/demo_tour.py` runs one deterministic cycle of all five scenarios in about thirty seconds. Repeated runs print identical numbers:

Scenario	Average risk	Worst node	Level	Alerts
normal	3.9	Greenwich	SAFE	0
flood	50.8	Regent's Canal (Little Venice)	CRITICAL	10
heatwave	78.8	Brixton	CRITICAL	12
smog	82.7	Brixton	CRITICAL	15
storm	98.1	Brixton	CRITICAL	20

Normal conditions stay quiet; each emergency escalates the nodes its hazard should hit, and every reading is labelled as simulated.

```

$ python scripts/judge_validate.py
>>> 1. Smoke test: scripts/smoke_test.py
[PASS] 20 nodes generate readings - got 20
[PASS] readings match canonical shape
[PASS] risk scores within 0-100
[PASS] risk levels valid
[PASS] dashboard can read latest readings
[PASS] dashboard can read risk scores
[PASS] simulation works without sensors
[PASS] hardware mode does not crash without sensors - No sensor driver library detected - simulation fallback active.
[PASS] node registry has 20 nodes
RESULT: 10/10 checks passed - PASS
No sensor driver library detected - simulation fallback active.

>>> 2. Pytest suite: -m pytest
..... [ 43%]
..... [ 86%]
106 passed in 18.53s

>>> 3. Demo cycle (normal): run.py --mode demo --scenario normal --once
[once] mode=demo scenario=normal source=demo nodes=20 avg_risk=4.4 alerts=0

>>> 4. Demo cycle (flood): run.py --mode demo --scenario flood --once
[once] mode=demo scenario=flood source=demo nodes=20 avg_risk=48.2 alerts=6

>>> 5. Evidence export: scripts/export_evidence.py
[evidence] Wrote 5 files to /tmp/claude-0/-home-user-Blank/Bef4c69-8083-5026-9a9d-07271c75f623/scratchpad/ClimateMesh-work/evidence
readings=48 risk_scores=48 alerts=0 nodes online=28
avg_risk=48.2 max_risk=100.0 training_mode=synthetic seed=1234 commit=3665f994
data source summary:
Source      Meaning      Count
hardware    real physical sensor      0
api          live Open-Meteo data      0
demo        deterministic demo / digital-twin data 40
simulation   generated offline data    0

Climate Mesh - Judge Validation
=====
Stop      Result  Detail
-----
1. Smoke test      PASS    10/10 checks
2. Pytest suite    PASS    106 passed
3. Demo cycle (normal)  PASS    nodes=20 avg_risk=4.4 alerts=0
4. Demo cycle (flood)  PASS    nodes=20 avg_risk=48.2 alerts=6
5. Evidence export  PASS    5 files
RESULT: PASS (5/5 steps passed)

```

Figure 4 — Output of `python scripts/judge_validate.py` on the submitted code.

7. The Raspberry Pi 5

The Pi 5 is the whole product: it runs the simulator or the live data source, the scikit-learn model, the risk engine, the database and the web dashboard at the same time, headless, on a few watts, at a site. Setup is one script (`setup_pi.sh`): it creates a virtual environment, installs the requirements (all prebuilt wheels, no compiler) and runs the smoke test. The dashboard listens on the Pi's loopback interface only and is viewed over an SSH tunnel, so nothing is exposed on the school network. The map has an offline basemap option, so even the Live Map works with no internet.

Bill of materials for one node (approximate UK prices, September 2026):

Item	Purpose	Approx. cost
Raspberry Pi 5 (4 GB)	Runs everything	£55
27 W USB-C power supply	Power	£12
32 GB microSD card	OS and software	£8
Case with fan	Protection	£10
Core node total		£85
Vernier Go Direct Weather (GDX-WTHR), optional	Physical temperature, humidity, wind and pressure	~£130

A community node therefore costs less than a hundred pounds before sensors; the software costs nothing and needs no subscription.

8. Impact

Climate Mesh is aimed at the people who actually respond first: a school site manager, a receptionist, a residents' association. It tells them a risk score they can read at a glance, *why* it is rising, and three practical things to do, such as clearing drains, opening a cooling room or moving outdoor PE. It stores only environmental measurements, never personal data, and runs entirely locally, so a school can adopt it without a data-protection review or a cloud bill. Because every node is cheap and every reading carries its provenance, a network of schools could pool their nodes into a genuine neighbourhood early-warning mesh.

9. Teamwork and what we learned

We are a team of two, Luis Yu and Leo Zhang.

► **Edit this section before submission so it is accurate.** Judges of the Inspiration Award look for a clear account of who did what. Suggested structure: one paragraph each on your main responsibilities (for example simulation and risk engine; dashboard and evidence tooling; hardware research; testing; write-up), then a paragraph on how you split the work and reviewed each other's code.

What was hardest. Making an honest system is harder than making an impressive one. Deciding that a reading may only be called "hardware" after a device physically opens, that a precipitation-derived water level must be flagged estimated, and that the AI must record whether it trained on real or synthetic data, shaped almost every module. Making the demo fully deterministic so that every screenshot can be reproduced was a second challenge. A third was testing a live dashboard: we now render all seven tabs in an automated browser harness on every test run, which caught a real crash.

What we learned. How anomaly detection differs from thresholds; why provenance matters in safety tooling; how to structure a Python project so that a hardware adapter, a simulator and a web API are interchangeable; how to write tests that pin published claims; and how to run a whole ML pipeline on a single-board computer.

10. What we would do next

1. **Validate one physical node.** Read a real Vernier GDX-WTHR through the existing adapter and compare it against the digital twin for the same location.
2. **Calibrate against history.** Use the evidence export to compare risk scores with recorded local flood and heat events, and tune thresholds from data instead of by hand.
3. **Physical mesh links.** Two or more real Pi nodes exchanging readings (Wi-Fi first, LoRa later) so mesh correlation runs across actual devices.
4. **Node cost pack.** A priced, tested bill of materials for a minimal community node, published once a real node has run on it.

11. Safety, privacy and responsible use

No personal data is stored: no names, accounts, cameras, microphones or tracking of people. The only optional outbound calls fetch public Open-Meteo weather data. Emergency scenarios are simulated and labelled as such, so judging never depends on or misrepresents a real emergency. Alerts are decision support and always defer to official emergency guidance.

12. Acknowledgements and tools

Built in Python with Streamlit (dashboard), scikit-learn (Isolation Forest), pandas, NumPy and Plotly, and SQLite. Live and archive weather data come from the free Open-Meteo API. The hardware path uses Vernier's godirect driver and.gdx helper. Early-warning statistics are from the World Meteorological Organization. Heat index follows the NOAA/Steadman formulation.

► **Edit:** if the competition asks entrants to declare other help, list here any mentors, tutorials or AI-assisted tools used during development.

Appendix A — Running it yourself

```
python3 -m venv .venv && source .venv/bin/activate # Windows:
.venv\Scripts\activate
pip install -r requirements.txt
python scripts/judge_validate.py # smoke test + 166 tests + 2 demo cycles +
export
python scripts/demo_tour.py # all five scenarios in ~30 s
python run.py --mode demo --scenario flood --judge-mode # terminal 1
python -m streamlit run dashboard/app.py # terminal 2 →
http://localhost:8501
```

Appendix B — Repository map

run.py	launcher: modes, scenarios, judge mode, --once
config/nodes.py	20 London nodes + mesh neighbour map (6 km)
sensors/	canonical reading contract + simulated / API / Vernier
adapters	

simulation/ coupling	data generation: daily cycle, noise, scenario deltas, mesh
ai/anomaly_model.py training)	explainable Isolation Forest (synthetic or ERA5-archive
backend/risk_engine.py	six sub-scores → 0–100, AI × mesh multipliers, alerts
backend/playbooks.py	community action playbooks per hazard
data/database.py	SQLite (WAL) message bus
dashboard/ scripts/ hardware read	seven-tab Streamlit dashboard with provenance badges judge_validate, demo_tour, export_evidence, smoke_test,
tests/ docs/screenshots/	166 automated tests reference screenshots and terminal evidence